Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет информационных технологий и программирования
Кафедра прикладной информатики

ДОМАШНЕЕ ЗАДАНИЕ №2

Проектирование и технический дизайн API

Выполнил: студент группы К3341
Кижваткин Н.М.

Санкт-Петербург
2025

1. ВВЕДЕНИЕ

Данный отчёт посвящён проектированию и техническому дизайну REST API системы Callmentor. API является интерфейсом взаимодействия между клиентскими приложениями (веб-фронтенд, внешние CRM-системы, Telegram-бот) и серверной частью платформы. Грамотное проектирование API обеспечивает корректность, предсказуемость поведения и удобство интеграции для разработчиков.

2. АНАЛИЗ ТРЕБОВАНИЙ К API

Система Callmentor предоставляет два уровня API: внутренний API (для собственного фронтенда, с JWT-аутентификацией) и внешний публичный API v1 (для интеграции со сторонними системами, с API-ключом). Оба варианта реализованы в рамках одного FastAPI-приложения gateway.

Функциональные требования к API:

- Управление звонками: загрузка аудиофайлов, получение статуса обработки, просмотр результатов.
- Управление промтами: создание, редактирование, удаление критериев оценки звонков.
- Аналитические дашборды: статистика по компании, менеджерам, скриптам.
- Аутентификация и авторизация: вход, регистрация, ролевое разграничение доступа.
- Интеграции: управление настройками CRM-интеграции.
- Отчёты: формирование и загрузка аналитических отчётов.

3. АРХИТЕКТУРА API

3.1. Технологический стек

API реализован с использованием фреймворка FastAPI (Python). FastAPI обеспечивает автоматическую генерацию OpenAPI-документации, валидацию входных данных на основе Pydantic-моделей, асинхронную обработку запросов через asyncio. Для документирования используется Scalar (вместо стандартного Swagger UI), доступный по эндпоинту /docs.

3.2. Структура маршрутизации

API организован по доменному принципу. Каждая функциональная область имеет собственный роутер. Все роутеры регистрируются в центральном api_router:

Таблица 1 — Структура роутеров API

Роутер	Префикс	Описание
auth_router	/auth	Аутентификация и авторизация
calls_router	/call	Управление звонками
dashboards_router	/dashboard	Аналитика и дашборды
prompts_router	/prompts	Управление промтами
reports_router	/reports	Формирование отчётов
artefacts_router	/artefacts	Артефакты анализа
integrations_router	/integrations	CRM-интеграции
admin_panel_router	/admin-panel	Панель администратора
service_admin_router	/service-admin	Суперадмин
ml_callback_router	/ml-callback	Колбэк от ML-сервисов
v1_router	/v1	Публичный API v1

4. ДЕТАЛЬНОЕ ОПИСАНИЕ ЭНДПОИНТОВ

4.1. Модуль аутентификации (/auth)

Таблица 2 — Эндпоинты аутентификации

Метод	Путь	Описание	Аутентификация
POST	/auth/login	Вход в систему (OAuth2 Password)	Публичный
POST	/auth/register	Регистрация пользователя	Публичный
GET	/auth/me	Данные текущего пользователя	Bearer JWT
POST	/auth/telegram/token/generate	Генерация TTL-токена для Telegram	Bearer JWT
POST	/auth/telegram/token/validate	Валидация токена Telegram	Публичный

4.2. Модуль управления звонками (/call)

Таблица 3 — Эндпоинты управления звонками

Метод	Путь	Описание
POST	/call/process	Загрузка аудио и создание задачи на анализ
GET	/call/check/{task_id}	Проверка статуса обработки звонка
GET	/call/	Получение всех звонков компании (пагинация)
GET	/call/manual-active	Звонки в статусах ожидания ручной отправки
POST	/call/manual-send/{task_id}	Ручной запуск обработки конкретного звонка

POST	/call/manual-send-batch	Массовый ручной запуск обработки
GET	/call/position	Позиция звонка на странице коммуникаций
GET	/call/statuses	Список всех возможных статусов
PATCH	/call/{task_id}	Обновление комментария к звонку
POST	/call/process-by-period	Загрузка звонков за период (из CRM)

4.3. Модуль дашборда (/dashboard)

Таблица 4 — Эндпоинты дашборда

Метод	Путь	Описание
POST	/dashboard/communications	Звонки с результатами анализа для страницы «Коммуникации»
GET	/dashboard/statistics	Краткая статистика за период для главной страницы
GET	/dashboard/stages-detailed	Детализация по стадиям скриптов
GET	/dashboard/managers-detailed	Детализация по менеджерам
GET	/dashboard/managers	Список менеджеров компании
GET	/dashboard/clients	Список клиентов
GET	/dashboard/phone-numbers	Список телефонных номеров
GET	/dashboard/first-date	Дата первого звонка компании

4.4. Публичный API v1 (/v1)

Версионированный публичный API предназначен для внешних интеграций. Аутентификация производится через API-ключ (APIKeyHeader). Набор эндпоинтов включает: управление звонками, промтами, дашборды и отчёты — аналогично внутреннему API, но с отдельной OpenAPI-документацией по адресу /docs-user.

5. СХЕМА АУТЕНТИФИКАЦИИ И АВТОРИЗАЦИИ

5.1. Двойная схема аутентификации

Система поддерживает две параллельные схемы аутентификации:

1. OAuth2 Password Bearer (JWT) — для внутреннего фронтенда. Пользователь передаёт email и пароль через форму /auth/login, получает JWT-токен. Токен передаётся в заголовке Authorization: Bearer <token>. Срок действия настраивается через переменную TOKEN_EXPIRES_MINUTES.
2. API Key (APIKeyHeader) — для внешних интеграций через /v1. Ключ передаётся в заголовке Authorization. Каждая компания имеет уникальный api_token в таблице companies.

5.2. Ролевая модель (RBAC)

Доступ к эндпоинтам контролируется middleware PermissionMiddleware. Middleware проверяет принадлежность пользователя к роли, имеющей права на вызываемый маршрут. Это реализовано через связи: пользователь → роль (m2m_user_role_company) → маршрут (m2m_role_routes). Определённые эндпоинты являются публичными (auth/login, demo), административными (service-admin, integrations) или внутренними (ml-callback).

6. СТАНДАРТЫ ПРОЕКТИРОВАНИЯ

6.1. Форматы ответов

Все эндпоинты возвращают JSON-ответы. Успешные ответы следуют структуре Pydantic Response Models. Ошибочные ответы используют стандартизированный формат с полями status и detail. Для типовых кодов ошибок (400, 404, 500) определены шаблоны TemplatesResponsesDataclass.

6.2. Пагинация

Для эндпоинтов, возвращающих списки данных, применяется пагинация через query-параметры limit и offset. Ответ включает поле total_count для построения клиентской пагинации.

6.3. Версионирование

API поддерживает две версии одновременно: legacy (без версии) и v1. Версия v1 имеет отдельную OpenAPI-схему, доступную по /openapi-v1.json. Функция _build_v1_openapi фильтрует схему, оставляя только маршруты под префиксом /v1.

7. ВЫВОДЫ

В рамках данной работы спроектировано REST API системы Callmentor. API включает 12 функциональных роутеров, охватывающих все аспекты бизнес-логики системы. Реализована двойная схема аутентификации (JWT и API-ключ), ролевая модель доступа и версионирование API. Архитектурное разделение на внутренний и публичный API обеспечивает гибкость интеграции и безопасность системы.